Day 01

Full Stack Development

Internship Task Documentation

Topic	Introduction to Full Stack Development
Prepared by	Sakshi Ramesh Karale
Date	14 October 2026

1. What is Full Stack Development?

Full Stack Development is the process of developing a complete web application by working on the frontend, backend, and database.

A Full Stack Developer works on the user interface, server-side logic, and database management to create a fully functional web application.

Example: An online shopping website where users can view products, add items to a cart, and place orders.

2. Frontend vs Backend vs Database

Frontend

The frontend is the visible part of a website with which users interact. It includes buttons, forms, images, menus, and page layouts.

Technologies: HTML, CSS, JavaScript, React.js.

Backend

The backend handles application logic, processes user requests, performs operations, and communicates with the database.

Technologies: Python, Django, Django REST Framework.

Database

A database stores, organizes, retrieves, and manages application data such as user details, product information, and orders.

Technologies: MySQL and SQL.

Component	Purpose	Technologies
Frontend	User interface	HTML, CSS, JavaScript, React.js
Backend	Application logic	Python, Django, Django REST Framework
Database	Data storage and management	MySQL, SQL

3. Client-Server Architecture

Client-server architecture is a model in which a client sends a request to a server, and the server processes the request and returns a response.

Architecture Diagram :

CLIENT / FRONTEND

HTML, CSS, JavaScript, React.js

↓ HTTP Request

BACKEND / SERVER

Python, Django, Django REST Framework

↓ Database Query

DATABASE

MySQL / SQL

Communication Flow:

- The client sends a request to the server.
- The server processes the request.
- The server communicates with the database when required.
- The database returns the requested information.
- The server sends a response to the client.
- The client displays the result to the user.

4. HTTP Request-Response Cycle

The HTTP request-response cycle is the process through which a client and a web server communicate using HTTP.

Steps

1. The user opens a website or performs an action.
2. The frontend sends an HTTP request to the server.
3. The server receives and processes the request.
4. The backend communicates with the database if required.
5. The server sends an HTTP response to the client.
6. The frontend displays the response to the user.

Example: When a user opens a website, the browser sends a request to the server, and the server returns the webpage.

5. What is a REST API?

REST API stands for Representational State Transfer Application Programming Interface. It allows different software applications to communicate and exchange data through HTTP requests.

A REST API commonly connects the frontend with the backend. The frontend sends requests to the backend API, and the backend returns the required data.

Common HTTP Methods

Method	Purpose	Example
GET	Retrieves data	Display product details
POST	Creates or submits data	Add a new order
PUT	Updates existing data	Update user information
DELETE	Deletes data	Delete a product record

Example: In an online shopping application, the frontend uses a REST API to retrieve product details from the backend.

6. Role of a Database

A database is an essential component of a web application because it stores and manages information.

Main Functions

- Stores user details, product information, and order records.
- Retrieves data when the application needs it.
- Supports inserting, updating, and deleting records.
- Organizes data for efficient access and management.
- Helps maintain consistency of application information.

MySQL is a relational database management system, while **SQL** is used to work with relational databases.

7. Full Stack Developer Workflow

A Full Stack Developer follows a series of steps to develop a web application.

1. **Requirement Analysis:** Understand the project requirements and features.
2. **UI Design:** Plan the user interface and page layouts.

3. **Frontend Development:** Build the interface using HTML, CSS, JavaScript, and React.js.
4. **Backend Development:** Implement application logic using Python and Django.
5. **Database Integration:** Create database tables and connect the backend to the database.
6. **Testing and Debugging:** Test the application and fix errors.
7. **Deployment:** Make the application available to users.
8. **Maintenance:** Update the application and resolve issues when necessary.

8. Introduction to Python Full Stack Development

Python Full Stack Development involves building complete web applications using frontend technologies, Python-based backend frameworks, and database technologies.

It allows developers to create user interfaces, implement server-side logic, and manage application data.

Technologies Used

Technology	Purpose
HTML	Defines the structure of web pages
CSS	Styles web pages and controls layout
JavaScript	Adds interactivity to web pages
React.js	Builds reusable user interface components
Python	Implements backend logic
Django	Develops server-side web applications
Django REST Framework	Helps create REST APIs
SQL	Queries and manages relational data
MySQL	Stores and manages relational data

These technologies work together to build complete and interactive web applications.

9. Real-World Application Example: Online Shopping Website

An online shopping website is a real-world example of Full Stack Development.

Users can browse products, view prices, add items to a shopping cart, and place orders.

Components

Frontend:

- Displays products, prices, and images.
- Provides search options and a shopping cart.
- Allows users to interact with the website.

Backend:

- Processes user requests.
- Handles login and order operations.
- Applies business logic.

Database:

- Stores customer details.
- Stores product information and prices.
- Maintains order records and inventory information.

REST API:

- Transfers data between the frontend and backend.
- Helps retrieve product details and submit orders.

How It Works

1. The user opens the online shopping website.
2. The frontend displays available products.
3. The user selects a product.
4. The frontend requests product information from the backend.
5. The backend retrieves the relevant data from the database.
6. The backend returns the response through the API.
7. The frontend displays the product details to the user.

This example shows how the frontend, backend, database, and REST API work together.

10. Conclusion

Full Stack Development combines frontend development, backend programming, and database management to build complete web applications.

Client-server architecture explains how clients and servers communicate. The HTTP request-response cycle describes how requests and responses are exchanged, while REST APIs enable communication between different application components.

Technologies such as HTML, CSS, JavaScript, React.js, Python, Django, Django REST Framework, SQL, and MySQL can be used together to develop modern web applications.

Expected Output / Deliverables

1. A document explaining Full Stack Development fundamentals.
2. A basic client-server architecture diagram.
3. An explanation of the HTTP request-response cycle.
4. An overview of REST APIs and HTTP methods.
5. An explanation of the database role.
6. An overview of Python Full Stack technologies.
7. A real-world application example.